حساب

پول‌هایت، روشن و مرتب.

طراحی و پیاده‌سازی سامانه

مدیریت مالی شخصی

فارسی و شمسی

محلی و تک‌کاربره

یک اصل در طراحی

همه موجودی،
قابل خرج نیست.



قابل خرج ← موجودی → ذخیره هدف‌ها

نمایش مفهومی؛ نسبت نوار، داده مالی واقعی نیست.

دانشجو
[نام و نام خانوادگی]

استاد
[نام استاد]

شماره دانشجویی
[شماره دانشجویی]

عنوان درس
[عنوان درس]

شماره ترم / نیمسال
[شماره ترم / نیمسال تحصیلی]

تاریخ ارائه
[تاریخ ارائه شمسی]

چرا «حساب»؟

از ثبت پراکنده هزینه‌ها به یک تصویر قابل فهم از پول.

ثبت روزمره

درآمد و هزینه با تاریخ شمسی، دسته و توضیح در یک دفتر محلی.

کنترل ماه

مقایسه مصرف دسته‌ها با سقف بودجه و دیدن اضافه‌خرج.

دیدن آینده

تفکیک پول قابل خرج از مبلغی که برای هدف‌ها کنار گذاشته‌ایم.

یک کاربر، یک موجودی، تصمیم‌های مالی روشن‌تر.

مسیر ارائه: محصول ← قواعد مالی ← معماری و داده ← ارزیابی

کوچک، اما با چرخه کامل

در این نسخه

- ثبت، اصلاح و حذف تراکنش
- بودجه تکرارشونده و استثنای ماهانه
- هدف، تخصیص و تاریخچه
- CSV و پشتیبان کامل

مرزهای آگاهانه

- بدون بانک، ورود کاربر یا ابر
- بدون چند حساب و چند ارز
- بدون ورود CSV یا تراکنش آینده
- بدون رمزگذاری فایل‌ها

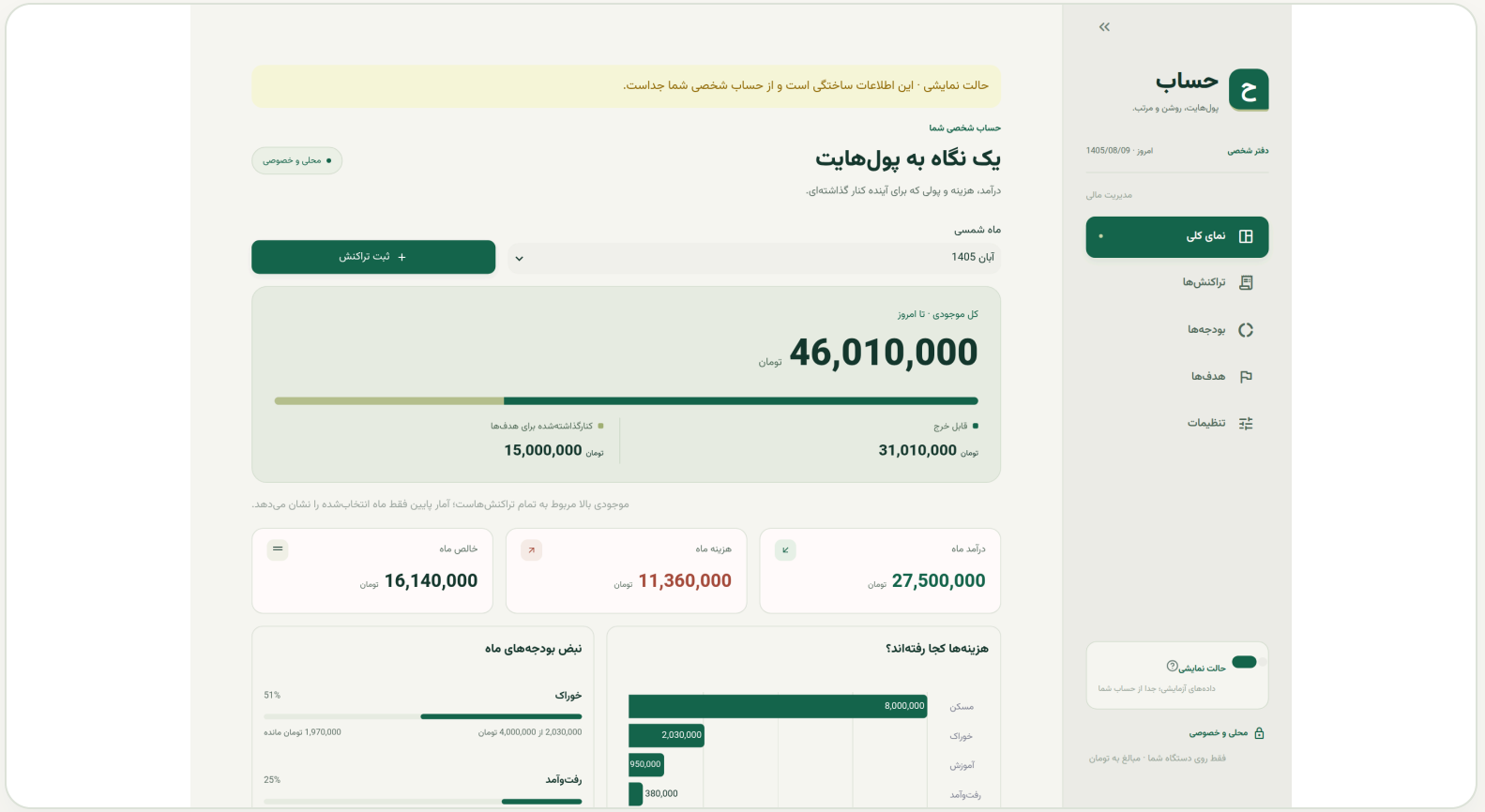
Python 3.12+ · Streamlit · SQLite · Plotly · jdatetime · uv

اول موجودی؛ بعد جزئیات ماه

سه لایه اطلاعات

- کل موجودی و نحوه تخصیص
- درآمد، هزینه و خالص ماه
- نمودار دسته‌ها و وضعیت بودجه

> تغییر ماه، آمار ماه را عوض می‌کند؛ نه موجودی کل را.



تصویر واقعی برنامه · فقط داده ساختگی

پس‌انداز، هزینه نیست.

تخصیص به هدف فقط پول موجود را رزرو می‌کند.

رویداد نمونه	کل موجودی	ذخیره	قابل خرج
موجودی اولیه	10,000,000	0	10,000,000
تخصیص 4,000,000 به هدف	10,000,000	4,000,000	6,000,000
ثبت هزینه 1,000,000	9,000,000	4,000,000	5,000,000

قابل خرج = موجودی کل - ذخیره هدف‌ها

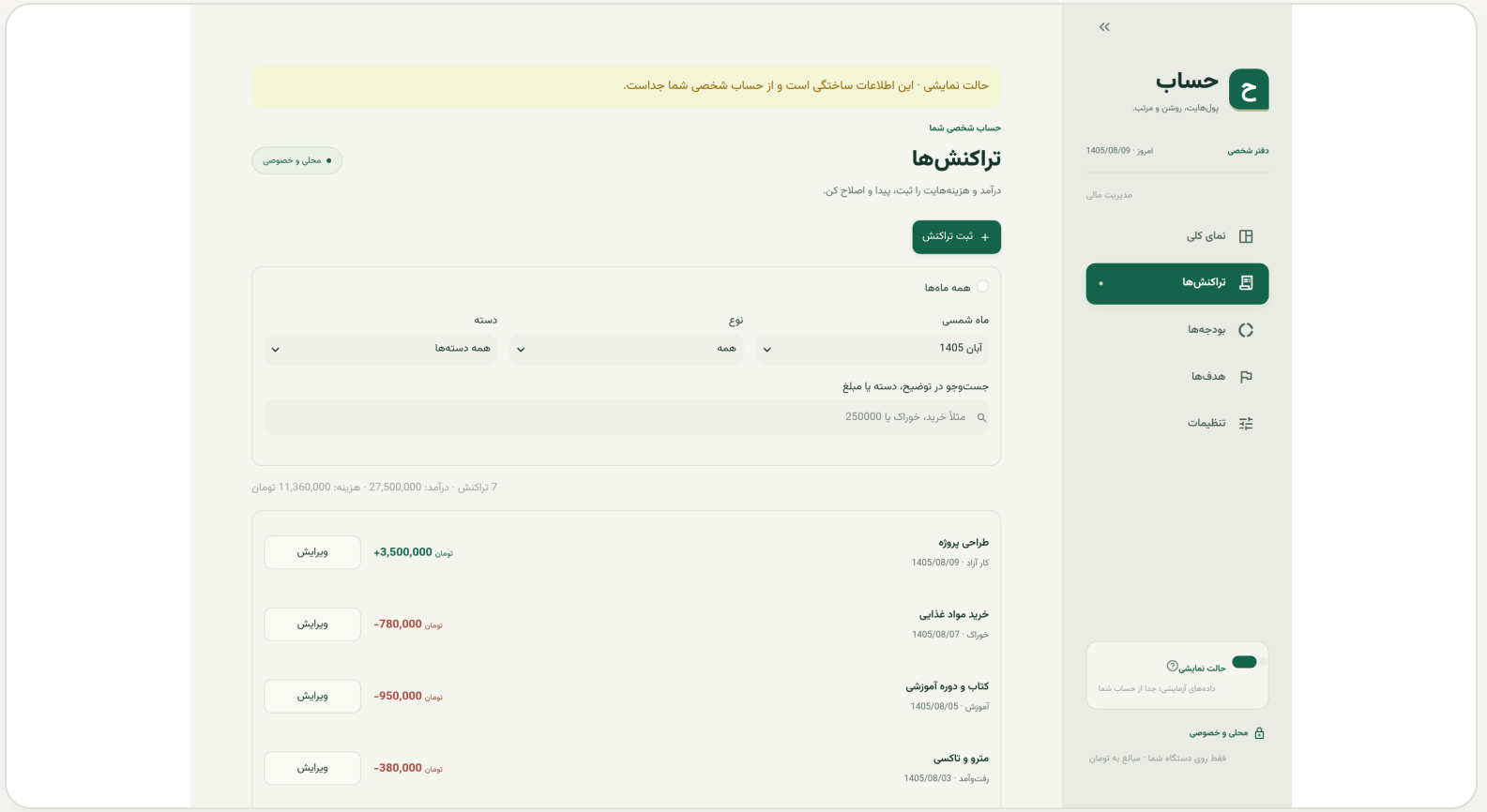
افتتاحیه درآمد نیست؛ خرید واقعی باید جداگانه به‌عنوان هزینه ثبت شود.

ثبت ساده، اصلاح قابل کنترل

قواعد ثبت

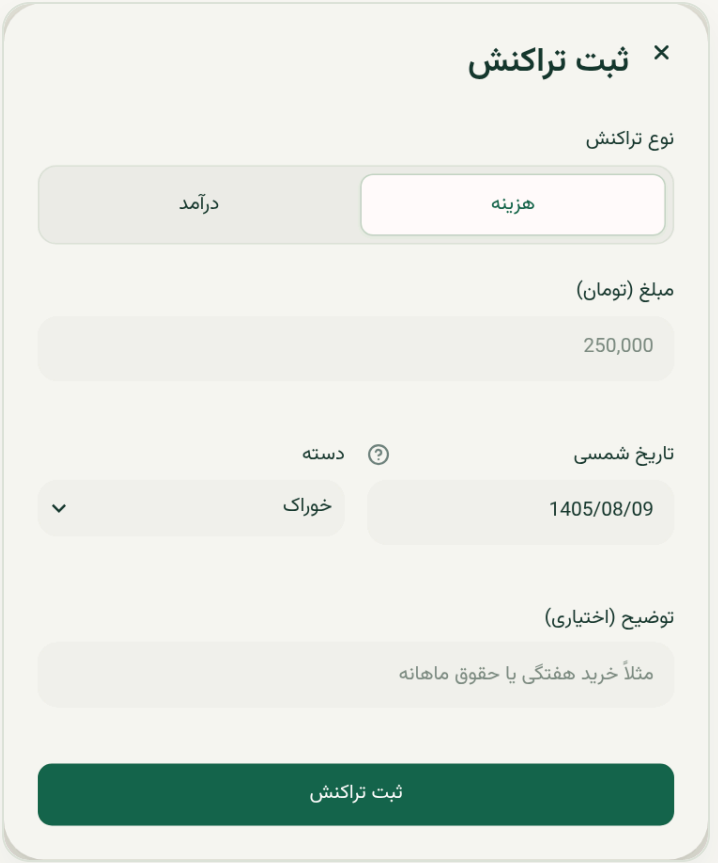
- مبلغ مثبت و صحیح، به تومان
- تاریخ از شروع حساب تا امروز
- دسته متناسب با نوع
- حذف فقط با تأیید کاربر

کسری موجودی هشدار می‌دهد؛ ثبت واقعیت را ممنوع نمی‌کند.



فیلتر ماه، نوع و دسته؛ جست‌وجوی توضیح، دسته یا مبلغ

از ورودی فارسی تا داده معتبر



مرز تبدیل و اعتبارسنجی

- `amount_value()` : تبدیل ارقام به integer تومان
- `parse_jalali()` : تاریخ شمسی به date
- `Store` : کنترل بازه تاریخ و نوع دسته
- SQL پارامتری؛ بدون الحاق متن کاربر

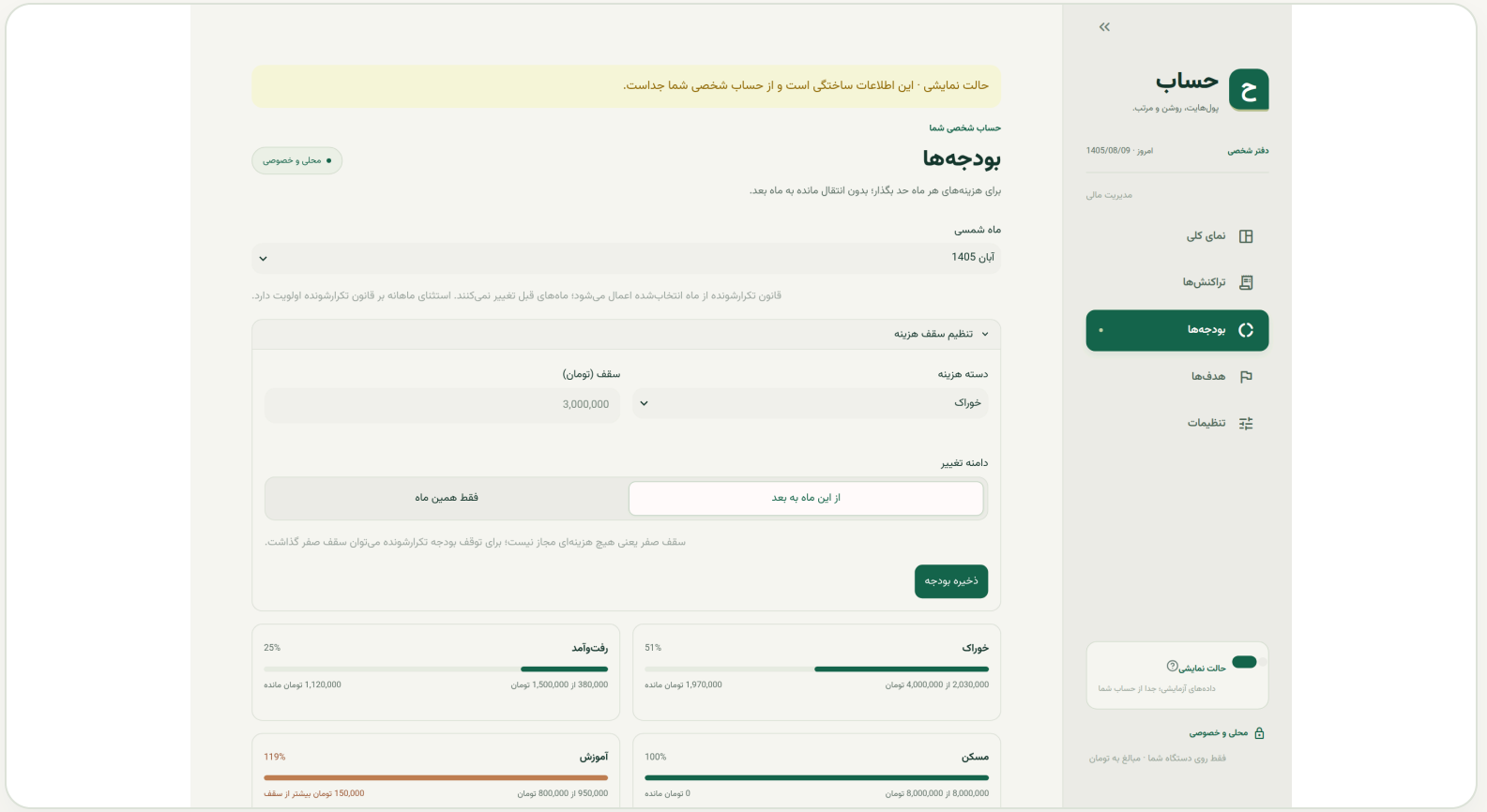
> رابط، ورودی را تبدیل می‌کند؛ Store قواعد مالی را اعمال می‌کند.

سقف ماه چگونه انتخاب می‌شود؟

الگوریتم انتخاب سقف

1. استثنای دقیق دسته و ماه
2. در نبود استثنا: آخرین قانون با `effective_month ≤ month`
3. در نبود قانون: بدون سقف

> کلید مرکب دسته و ماه؛ مقدار صفر معتبر است، نبود رکورد یعنی بدون بودجه.



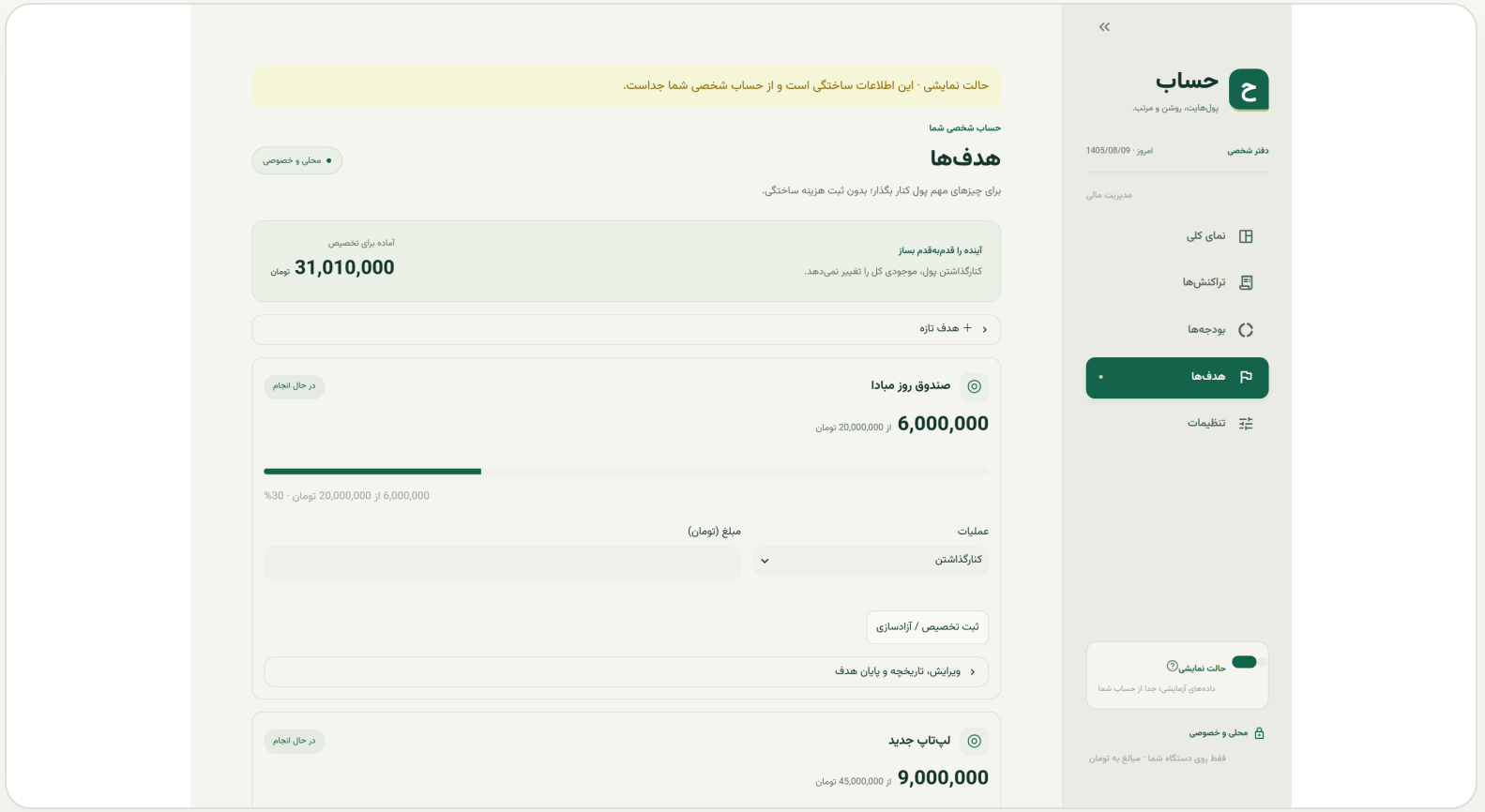
سقف نظارتی هزینه؛ نه برداشت یا تخصیص واقعی پول

مانده هدف، حاصل جمع رویدادهاست.

مدل ذخیره‌سازی

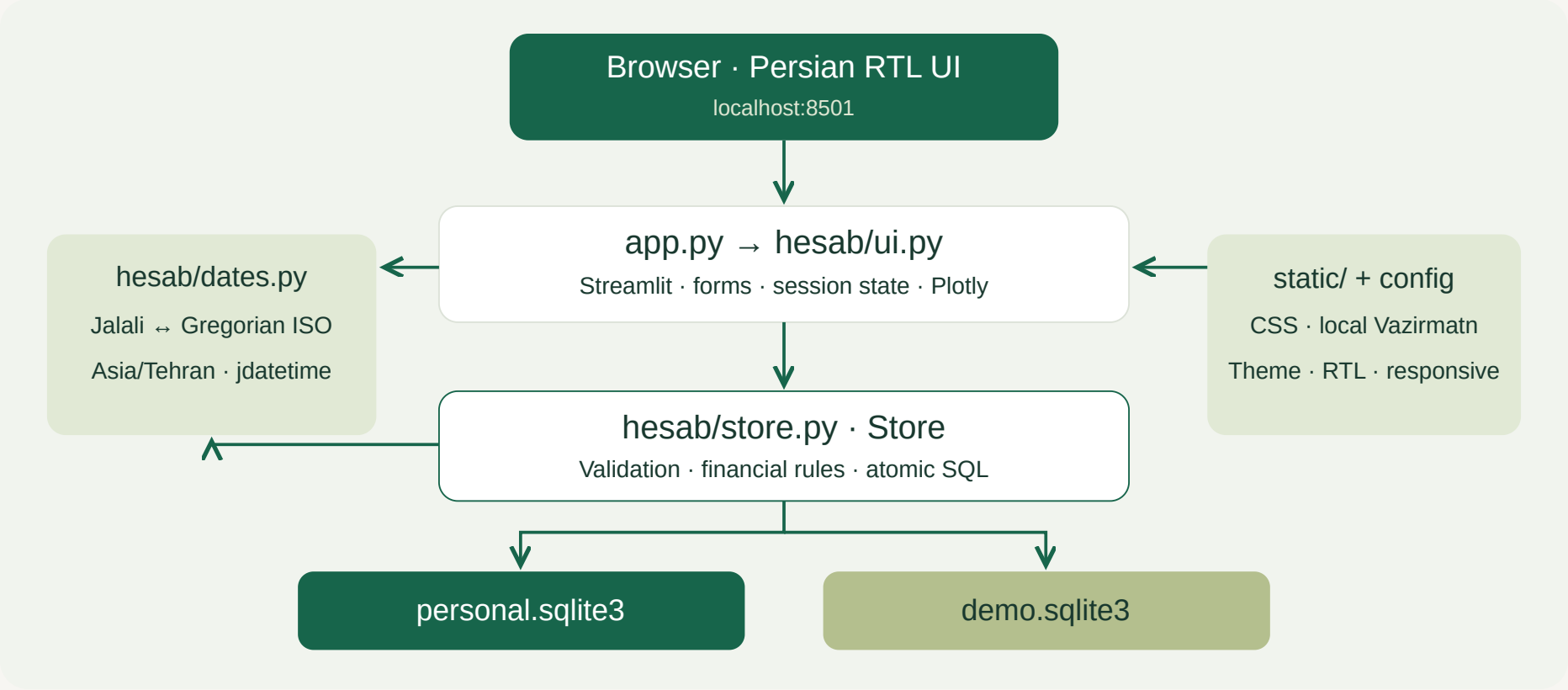
- تخصیص: رکورد با مبلغ مثبت
- آزادسازی: رکورد با مبلغ منفی
- ذخیره فعلی: `SUM(amount)`
- پایان هدف فقط با ذخیره صفر

ستون مانده جدا نداریم؛ تاریخچه منبع محاسبه است.



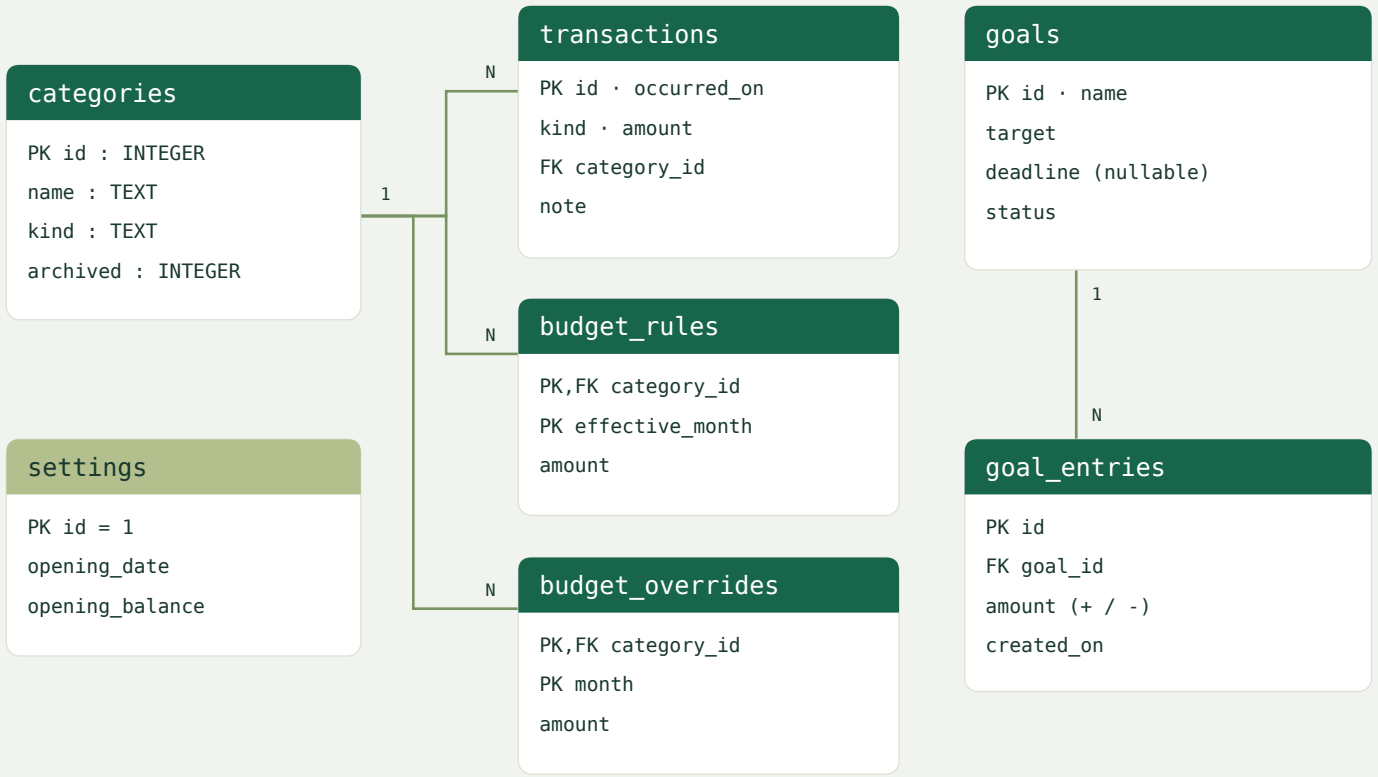
پیشرفت، مهلت اختیاری، تخصیص و آزادسازی

رابط سبک؛ منطق مالی مستقل



Streamlit فقط رابط و رویدادها را اداره می‌کند؛ تمام عملیات مالی از Store عبور می‌کنند.

هفت جدول با مسئولیت‌های مشخص



FK برای رابطه‌ها · CHECK برای علامت مبلغ · PK مرکب برای دسته/ماه؛ قواعد ترکیبی در Store کنترل می‌شوند.

تاریخ بیرونی شمسی؛ ذخیره داخلی ISO

تاریخ روز

ورودی و نمایش شمسی؛ ذخیره میلادی با قالب `YYYY-MM-DD`.

کلید بودجه

ماه شمسی با قالب `YYYY-MM`؛ انتخاب جدیدترین قانون معتبر.

مرز گزارش

از ابتدای ماه تا پیش از ابتدای ماه بعد؛ سازگار با اسفند کبیسه.

```
occurred_on >= start_iso
AND occurred_on < next_month_start_iso
```

امروز بر اساس Asia/Tehran محاسبه می‌شود. قیود CHECK و FK همراه اعتبارسنجی لایه برنامه اعمال می‌شوند.

یک موجودی، دو تخصیص هم‌زمان؟

موجودی قابل خرج 10,000 تومان؛ دو درخواست 8,000 تومانی.

بدون تراکنش مشترک

هر دو درخواست ممکن است موجودی قدیمی را ببینند و جمعاً 16,000 تومان رزرو کنند.

در حساب

بررسی و درج در یک BEGIN IMMEDIATE؛ درخواست دوم مقدار جدید را می‌بیند و رد می‌شود.

بررسی → ثبت → commit؛ یا rollback کامل

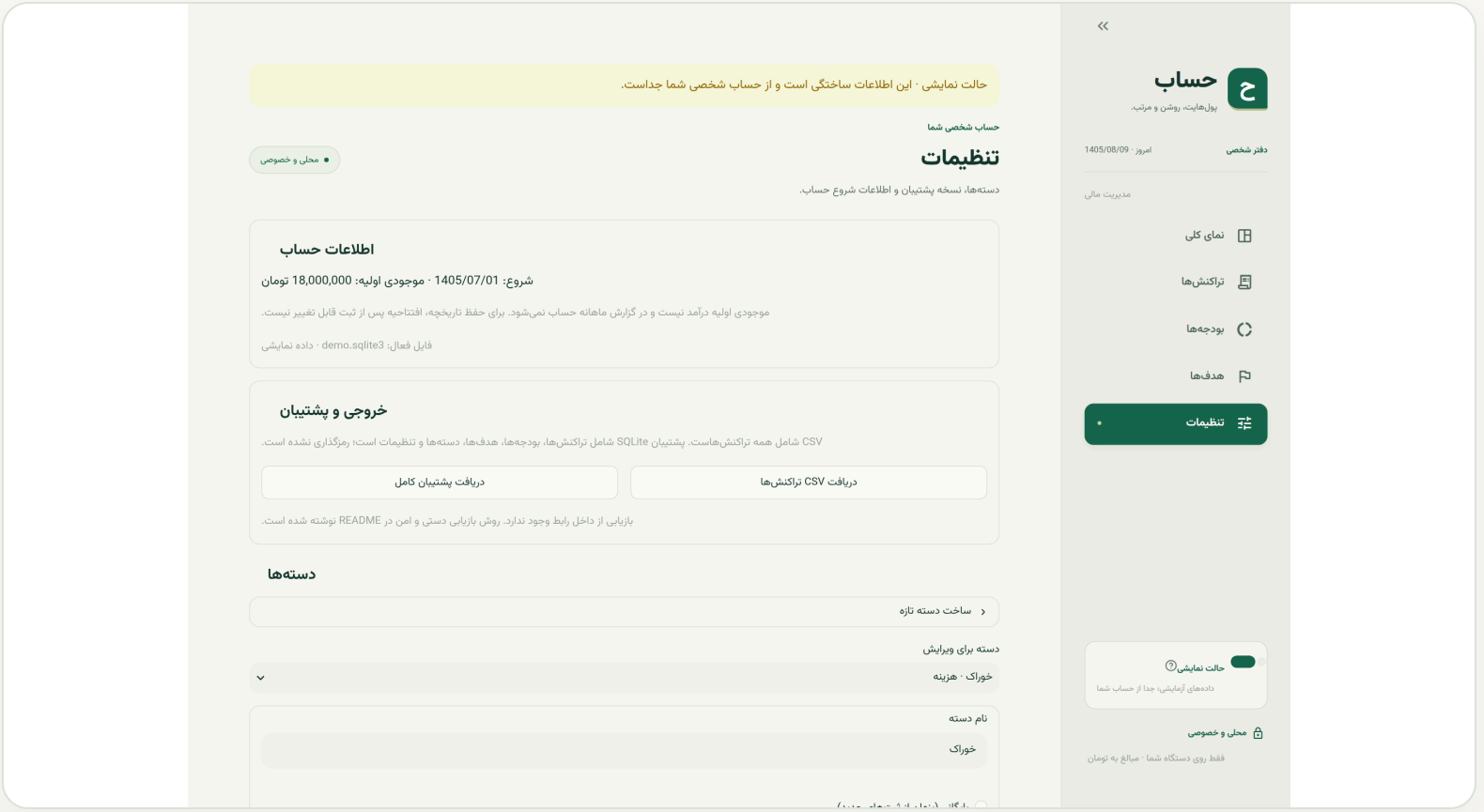
کلیدهای خارجی در هر اتصال فعال‌اند. این طراحی برای استفاده محلی است، نه نوشتن پرترافیک چندکاربره.

داده نزد کاربر می‌ماند.

دو خروجی، دو هدف

- CSV: همه تراکنش‌ها با تاریخ شمسی
- SQLite: کل حساب و هفت جدول
- Snapshot با API پشتیبان SQLite

> بدون احراز هویت و رمزگذاری؛ فقط برای localhost.



فایل‌های شخصی و نمایشی مستقل‌اند.

قواعد آزموده‌شده؛ ظاهر بررسی‌شده

25 / 25

آزمون موفق

قواعد مالی، تاریخ و جریان‌های رابط در pytest.

90.78%

پوشش مالی و تاریخ

فقط store و dates؛ نه پوشش کل UI.

1366 · 768 · 390

عرض‌های بررسی‌شده

چیدمان، ورودی، دکمه، گزینه‌ها و نوار کناری.

سناریوهای حساس: اسفند کبیسه، استثنای بودجه، کسری موجودی، تخصیص هم‌زمان و سلامت پشتیبان.

آزمون بار و ممیزی امنیتی مستقل انجام نشده‌اند؛ پوشش بالا تضمین نبود خطا نیست.

از ارسال فرم تا بازسازی صفحه

1. فرم

st.form ورودی‌ها را تا ارسال گروه‌بندی می‌کند.

2. اعتبارسنجی

تبدیل مبلغ و تاریخ؛ سپس بررسی قواعد در Store.

3. تراکنش

SQL پارامتری؛ commit موفق یا rollback خطا.

4. بازاجرا

st.rerun داده جدید را می‌خواند و صفحه را می‌سازد.

session_state

وضعیت موقت رابط: انتخاب صفحه، پیش‌نویس و پیام موفقیت.

SQLite

منبع دائمی اطلاعات مالی؛ مستقل از بازاجرا و بستن مرورگر.

بازاجرای رابط، تکرار خودکار ثبت نیست؛ نوشتن داده فقط در شاخه ارسال فرم انجام می‌شود.

گام بعدی، بر اساس نیاز واقعی

محدودیت فعلی

- فایل‌ها رمزگذاری نشده‌اند
- بازیابی فقط دستی است
- صفحه‌بندی پس از بارگذاری داده
- لاگ ممیزی تراکنش‌ها وجود ندارد

پیشنهاد توسعه

- مهاجرت نسخه‌ای ساختار داده
- بازیابی هدایت‌شده و اعتبارسنجی فایل
- بهینه‌سازی SQL با سنجش داده بزرگ
- آزمون دسترس‌پذیری و تجربه کاربر

این موارد مسیر پیشنهادی‌اند؛ قابلیت موجود یا تعهد پیاده‌سازی‌شده نیستند.

جمع‌بندی

حساب؛ از عدد به تصمیم.

یک چرخه کامل مدیریت مالی شخصی، با رابط فارسی و قواعد قابل توضیح و آزمون.

سه تصمیم اصلی

تفکیک ذخیره از هزینه · بودجه نسخه‌دار · عملیات مالی اتمیک

ارائه‌دهنده
[نام و نام خانوادگی]

درس
[عنوان درس]

استاد
[نام استاد]

سپاس از توجه شما — پرسش‌ها؟